

## Extend Unit UVM Verification

Generated by Doxygen 1.16.1



|                                         |           |
|-----------------------------------------|-----------|
| <b>1 Topic Index</b>                    | <b>1</b>  |
| 1.1 Topics                              | 1         |
| <b>2 Directory Hierarchy</b>            | <b>3</b>  |
| 2.1 Directories                         | 3         |
| <b>3 Hierarchical Index</b>             | <b>5</b>  |
| 3.1 Class Hierarchy                     | 5         |
| <b>4 Class Index</b>                    | <b>7</b>  |
| 4.1 Class List                          | 7         |
| <b>5 File Index</b>                     | <b>9</b>  |
| 5.1 File List                           | 9         |
| <b>6 Topic Documentation</b>            | <b>11</b> |
| 6.1 RTL design                          | 11        |
| 6.1.1 Detailed Description              | 11        |
| 6.2 Modules                             | 11        |
| 6.3 Testbench components                | 11        |
| 6.3.1 Detailed Description              | 11        |
| 6.4 Interfaces                          | 12        |
| 6.4.1 Detailed Description              | 12        |
| 6.4.2 Function Documentation            | 12        |
| 6.4.2.1 s_interface()                   | 12        |
| 6.5 UVM verification components         | 12        |
| 6.5.1 Detailed Description              | 13        |
| <b>7 Directory Documentation</b>        | <b>15</b> |
| 7.1 filter Directory Reference          | 15        |
| 7.1.1 Detailed Description              | 15        |
| 7.1.2 SystemVerilog Doxygen filter      | 15        |
| 7.2 reference_model Directory Reference | 15        |
| 7.3 rtl Directory Reference             | 15        |
| 7.4 tb Directory Reference              | 16        |
| 7.5 uvm Directory Reference             | 16        |
| <b>8 Class Documentation</b>            | <b>17</b> |
| 8.1 agent Class Reference               | 17        |
| 8.1.1 Detailed Description              | 18        |
| 8.1.2 Member Function Documentation     | 18        |
| 8.1.2.1 build_phase()                   | 18        |
| 8.1.2.2 connect_phase()                 | 18        |
| 8.1.2.3 new()                           | 18        |
| 8.1.2.4 uvm_component_utils()           | 19        |

|                                          |    |
|------------------------------------------|----|
| 8.1.3 Member Data Documentation          | 19 |
| 8.1.3.1 m_monitor                        | 19 |
| 8.1.3.2 m_sequencer                      | 19 |
| 8.2 coverage Class Reference             | 19 |
| 8.2.1 Detailed Description               | 20 |
| 8.2.2 Member Function Documentation      | 20 |
| 8.2.2.1 cg_extend_unit()                 | 20 |
| 8.2.2.2 new()                            | 21 |
| 8.2.2.3 uvm_component_utils()            | 21 |
| 8.2.2.4 write()                          | 21 |
| 8.3 driver Class Reference               | 21 |
| 8.3.1 Detailed Description               | 22 |
| 8.3.2 Member Function Documentation      | 23 |
| 8.3.2.1 build_phase()                    | 23 |
| 8.3.2.2 drive_item()                     | 23 |
| 8.3.2.3 new()                            | 23 |
| 8.3.2.4 run_phase()                      | 23 |
| 8.3.2.5 uvm_component_utils()            | 23 |
| 8.3.3 Member Data Documentation          | 23 |
| 8.3.3.1 req_pkt                          | 23 |
| 8.4 env Class Reference                  | 24 |
| 8.4.1 Detailed Description               | 25 |
| 8.4.2 Member Function Documentation      | 25 |
| 8.4.2.1 build_phase()                    | 25 |
| 8.4.2.2 connect_phase()                  | 25 |
| 8.4.2.3 new()                            | 25 |
| 8.4.2.4 uvm_component_utils()            | 25 |
| 8.4.3 Member Data Documentation          | 25 |
| 8.4.3.1 m_coverage                       | 25 |
| 8.4.3.2 m_scoreboard                     | 26 |
| 8.5 extend_unit_sequence Class Reference | 26 |
| 8.5.1 Detailed Description               | 27 |
| 8.5.2 Member Function Documentation      | 27 |
| 8.5.2.1 body()                           | 27 |
| 8.5.2.2 new()                            | 27 |
| 8.5.2.3 uvm_object_utils()               | 27 |
| 8.5.3 Member Data Documentation          | 27 |
| 8.5.3.1 num_packets                      | 27 |
| 8.6 monitor Class Reference              | 28 |
| 8.6.1 Detailed Description               | 29 |
| 8.6.2 Member Function Documentation      | 29 |
| 8.6.2.1 build_phase()                    | 29 |

|                                      |    |
|--------------------------------------|----|
| 8.6.2.2 new()                        | 29 |
| 8.6.2.3 run_phase()                  | 29 |
| 8.6.2.4 uvm_component_utils()        | 29 |
| 8.6.3 Member Data Documentation      | 29 |
| 8.6.3.1 mon_analysis_port            | 29 |
| 8.7 pkt Class Reference              | 30 |
| 8.7.1 Detailed Description           | 31 |
| 8.7.2 Member Function Documentation  | 31 |
| 8.7.2.1 new()                        | 31 |
| 8.7.2.2 uvm_field_int()              | 31 |
| 8.7.2.3 uvm_object_utils_begin()     | 31 |
| 8.7.3 Member Data Documentation      | 31 |
| 8.7.3.1 i_imm_ID                     | 31 |
| 8.7.3.2 i_imm_src_ID                 | 31 |
| 8.7.3.3 o_imm_ex_ID                  | 31 |
| 8.8 s_interface Interface Reference  | 32 |
| 8.8.1 Detailed Description           | 32 |
| 8.9 scoreboard Class Reference       | 32 |
| 8.9.1 Detailed Description           | 33 |
| 8.9.2 Member Function Documentation  | 33 |
| 8.9.2.1 build_phase()                | 33 |
| 8.9.2.2 check_phase()                | 33 |
| 8.9.2.3 new()                        | 33 |
| 8.9.2.4 run_phase()                  | 34 |
| 8.9.2.5 uvm_component_utils()        | 34 |
| 8.9.2.6 write()                      | 34 |
| 8.9.3 Member Data Documentation      | 34 |
| 8.9.3.1 ap_imp                       | 34 |
| 8.9.3.2 num_errors                   | 34 |
| 8.10 sequencer Class Reference       | 35 |
| 8.10.1 Detailed Description          | 35 |
| 8.10.2 Member Function Documentation | 36 |
| 8.10.2.1 uvm_component_utils()       | 36 |
| 8.10.3 Member Data Documentation     | 36 |
| 8.10.3.1 parent                      | 36 |
| 8.11 test Class Reference            | 36 |
| 8.11.1 Detailed Description          | 37 |
| 8.11.2 Member Function Documentation | 37 |
| 8.11.2.1 build_phase()               | 37 |
| 8.11.2.2 end_of_elaboration_phase()  | 38 |
| 8.11.2.3 new()                       | 38 |
| 8.11.2.4 run_phase()                 | 38 |

|                                                      |           |
|------------------------------------------------------|-----------|
| 8.11.2.5 uvm_component_utils()                       | 38        |
| 8.11.3 Member Data Documentation                     | 38        |
| 8.11.3.1 m_sequence                                  | 38        |
| <b>9 File Documentation</b>                          | <b>39</b> |
| 9.1 filter/README.md File Reference                  | 39        |
| 9.2 reference_model/reference_model.c File Reference | 39        |
| 9.2.1 Detailed Description                           | 40        |
| 9.2.2 Enumeration Type Documentation                 | 40        |
| 9.2.2.1 imm_src_e                                    | 40        |
| 9.2.3 Function Documentation                         | 40        |
| 9.2.3.1 extend_unit_golden()                         | 40        |
| 9.3 rtl/extend_unit.v File Reference                 | 41        |
| 9.3.1 Function Documentation                         | 41        |
| 9.3.1.1 always()                                     | 41        |
| 9.3.2 Variable Documentation                         | 41        |
| 9.3.2.1 b0                                           | 41        |
| 9.3.2.2 B_type                                       | 41        |
| 9.3.2.3 default                                      | 41        |
| 9.3.2.4 i_imm_ID                                     | 41        |
| 9.3.2.5 i_imm_src_ID                                 | 42        |
| 9.3.2.6 l_type                                       | 42        |
| 9.3.2.7 J_type                                       | 42        |
| 9.3.2.8 o_imm_ex_ID                                  | 42        |
| 9.3.2.9 S_type                                       | 42        |
| 9.3.2.10 U_type                                      | 42        |
| 9.4 rtl/extend_unit_wrapper.sv File Reference        | 42        |
| 9.4.1 Detailed Description                           | 42        |
| 9.5 tb/s_interface.sv File Reference                 | 42        |
| 9.5.1 Detailed Description                           | 43        |
| 9.6 tb/tb_top.sv File Reference                      | 43        |
| 9.6.1 Detailed Description                           | 43        |
| 9.6.2 Function Documentation                         | 43        |
| 9.6.2.1 run_test()                                   | 43        |
| 9.6.2.2 U_EXTEND_UNIT_WRAPPER()                      | 43        |
| 9.6.2.3 vif()                                        | 44        |
| 9.6.3 Variable Documentation                         | 44        |
| 9.6.3.1 forever                                      | 44        |
| 9.6.3.2 rst                                          | 44        |
| 9.7 uvm/agent.sv File Reference                      | 44        |
| 9.7.1 Detailed Description                           | 45        |
| 9.8 agent.sv                                         | 45        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 9.9 uvm/coverage.sv File Reference . . . . .              | 45        |
| 9.9.1 Detailed Description . . . . .                      | 46        |
| 9.10 coverage.sv . . . . .                                | 46        |
| 9.11 uvm/driver.sv File Reference . . . . .               | 46        |
| 9.11.1 Detailed Description . . . . .                     | 47        |
| 9.12 driver.sv . . . . .                                  | 47        |
| 9.13 uvm/env.sv File Reference . . . . .                  | 48        |
| 9.13.1 Detailed Description . . . . .                     | 48        |
| 9.14 env.sv . . . . .                                     | 49        |
| 9.15 uvm/extend_unit_pkg.sv File Reference . . . . .      | 49        |
| 9.15.1 Detailed Description . . . . .                     | 49        |
| 9.16 uvm/extend_unit_sequence.sv File Reference . . . . . | 50        |
| 9.16.1 Detailed Description . . . . .                     | 50        |
| 9.17 extend_unit_sequence.sv . . . . .                    | 50        |
| 9.18 uvm/monitor.sv File Reference . . . . .              | 51        |
| 9.18.1 Detailed Description . . . . .                     | 51        |
| 9.19 monitor.sv . . . . .                                 | 51        |
| 9.20 uvm/pkt.sv File Reference . . . . .                  | 52        |
| 9.20.1 Detailed Description . . . . .                     | 52        |
| 9.21 pkt.sv . . . . .                                     | 53        |
| 9.22 uvm/scoreboard.sv File Reference . . . . .           | 53        |
| 9.22.1 Detailed Description . . . . .                     | 54        |
| 9.22.2 Function Documentation . . . . .                   | 54        |
| 9.22.2.1 extend_unit_golden() . . . . .                   | 54        |
| 9.23 scoreboard.sv . . . . .                              | 54        |
| 9.24 uvm/sequencer.sv File Reference . . . . .            | 55        |
| 9.24.1 Detailed Description . . . . .                     | 55        |
| 9.25 sequencer.sv . . . . .                               | 56        |
| 9.26 uvm/test.sv File Reference . . . . .                 | 56        |
| 9.26.1 Detailed Description . . . . .                     | 56        |
| 9.27 test.sv . . . . .                                    | 57        |
| <b>Index</b>                                              | <b>59</b> |



# Chapter 1

## Topic Index

### 1.1 Topics

Here is a list of all topics with brief descriptions:

|                                       |    |
|---------------------------------------|----|
| RTL design . . . . .                  | 11 |
| Modules . . . . .                     | 11 |
| Testbench components . . . . .        | 11 |
| Interfaces . . . . .                  | 12 |
| UVM verification components . . . . . | 12 |



## Chapter 2

# Directory Hierarchy

### 2.1 Directories

|                         |    |
|-------------------------|----|
| filter                  | 15 |
| reference_model         | 15 |
| reference_model.c       | 39 |
| rtl                     | 15 |
| extend_unit.v           | 41 |
| extend_unit_wrapper.sv  | 42 |
| tb                      | 16 |
| s_interface.sv          | 42 |
| tb_top.sv               | 43 |
| uvm                     | 16 |
| agent.sv                | 44 |
| coverage.sv             | 45 |
| driver.sv               | 46 |
| env.sv                  | 48 |
| extend_unit_pkg.sv      | 49 |
| extend_unit_sequence.sv | 50 |
| monitor.sv              | 51 |
| pkt.sv                  | 52 |
| scoreboard.sv           | 53 |
| sequencer.sv            | 55 |
| test.sv                 | 56 |



## Chapter 3

# Hierarchical Index

### 3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                      |    |
|----------------------|----|
| s_interface          | 32 |
| uvm_agent            |    |
| agent                | 17 |
| uvm_driver           |    |
| driver               | 21 |
| uvm_env              |    |
| env                  | 24 |
| uvm_monitor          |    |
| monitor              | 28 |
| uvm_scoreboard       |    |
| scoreboard           | 32 |
| uvm_sequence         |    |
| extend_unit_sequence | 26 |
| uvm_sequence_item    |    |
| pkt                  | 30 |
| uvm_sequencer        |    |
| sequencer            | 35 |
| uvm_subscriber       |    |
| coverage             | 19 |
| uvm_test             |    |
| test                 | 36 |



## Chapter 4

# Class Index

### 4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                      |                                                                                                     |    |
|--------------------------------------|-----------------------------------------------------------------------------------------------------|----|
| <a href="#">agent</a>                | UVM agent that connects the sequencer, driver, and monitor . . . . .                                | 17 |
| <a href="#">coverage</a>             | UVM coverage subscriber for extend-unit transaction sampling . . . . .                              | 19 |
| <a href="#">driver</a>               | UVM driver for extend-unit transactions . . . . .                                                   | 21 |
| <a href="#">env</a>                  | Top-level UVM environment for the extend-unit verification bench . . . . .                          | 24 |
| <a href="#">extend_unit_sequence</a> | UVM sequence that drives extend-unit transactions until coverage reaches the target . . . . .       | 26 |
| <a href="#">monitor</a>              | UVM monitor for extend-unit transaction observation . . . . .                                       | 28 |
| <a href="#">pkt</a>                  | Sequence item that models one extend-unit operation (stimulus + expected response fields) . . . . . | 30 |
| <a href="#">s_interface</a>          | Extend-unit stimulus/response interface shared by the DUT and the UVM environment . . . . .         | 32 |
| <a href="#">scoreboard</a>           | UVM scoreboard for extend-unit functional checking . . . . .                                        | 32 |
| <a href="#">sequencer</a>            | UVM sequencer for extend-unit transactions . . . . .                                                | 35 |
| <a href="#">test</a>                 | Top-level UVM test for the extend-unit verification bench . . . . .                                 | 36 |



# Chapter 5

## File Index

### 5.1 File List

Here is a list of all files with brief descriptions:

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| reference_model/ <a href="#">reference_model.c</a>                                     |    |
| Golden reference model for the extend unit, called from the UVM scoreboard via DPI-C   | 39 |
| rtl/ <a href="#">extend_unit.v</a>                                                     | 41 |
| rtl/ <a href="#">extend_unit_wrapper.sv</a>                                            |    |
| Thin combinational wrapper around <a href="#">extend_unit</a> , used as the DUT        | 42 |
| tb/ <a href="#">s_interface.sv</a>                                                     |    |
| Extend-unit stimulus/response interface shared by the DUT and the UVM environment      | 42 |
| tb/ <a href="#">tb_top.sv</a>                                                          |    |
| Top-level testbench that instantiates the DUT, interface, and UVM test                 | 43 |
| uvm/ <a href="#">agent.sv</a>                                                          |    |
| UVM agent that bundles the sequencer, driver, and monitor                              | 44 |
| uvm/ <a href="#">coverage.sv</a>                                                       |    |
| Coverage collector for extend-unit transactions                                        | 45 |
| uvm/ <a href="#">driver.sv</a>                                                         |    |
| UVM driver that translates extend-unit transactions into interface signal activity     | 46 |
| uvm/ <a href="#">env.sv</a>                                                            |    |
| UVM environment that instantiates the agent, scoreboard, and coverage collector        | 48 |
| uvm/ <a href="#">extend_unit_pkg.sv</a>                                                |    |
| Package that includes the UVM classes used by the extend-unit verification environment | 49 |
| uvm/ <a href="#">extend_unit_sequence.sv</a>                                           |    |
| UVM sequence that drives extend-unit transactions until coverage reaches the target    | 50 |
| uvm/ <a href="#">monitor.sv</a>                                                        |    |
| UVM monitor that samples extend-unit transactions and publishes them to subscribers    | 51 |
| uvm/ <a href="#">pkt.sv</a>                                                            |    |
| Transaction item used to model one extend-unit operation in the UVM environment        | 52 |
| uvm/ <a href="#">scoreboard.sv</a>                                                     |    |
| Scoreboard that checks extend-unit transactions against the golden reference model     | 53 |
| uvm/ <a href="#">sequencer.sv</a>                                                      |    |
| UVM sequencer that provides transactions to the driver                                 | 55 |
| uvm/ <a href="#">test.sv</a>                                                           |    |
| Top-level UVM test that builds the environment and starts the main sequence            | 56 |



# Chapter 6

## Topic Documentation

### 6.1 RTL design

Register-transfer-level implementation of the extend-unit DUT.

#### Files

- file [extend\\_unit\\_wrapper.sv](#)  
*Thin combinational wrapper around [extend\\_unit](#), used as the DUT.*

#### 6.1.1 Detailed Description

Register-transfer-level implementation of the extend-unit DUT.

### 6.2 Modules

### 6.3 Testbench components

Interface and top-level testbench modules used by the verification environment.

#### Files

- file [s\\_interface.sv](#)  
*Extend-unit stimulus/response interface shared by the DUT and the UVM environment.*
- file [tb\\_top.sv](#)  
*Top-level testbench that instantiates the DUT, interface, and UVM test.*

#### 6.3.1 Detailed Description

Interface and top-level testbench modules used by the verification environment.

## 6.4 Interfaces

### Functions

- interface [s\\_interface](#) (input logic clk, input logic reset)

### 6.4.1 Detailed Description

### 6.4.2 Function Documentation

#### 6.4.2.1 s\_interface()

```
interface s_interface (
    input logic clk,
    input logic reset)
```

< Immediate format selector ([i\\_imm\\_src\\_ID](#)), driven by the driver.

< Raw instr[31:7] slice ([i\\_imm\\_ID](#)), driven by the driver.

< Sign-extended immediate ([o\\_imm\\_ex\\_ID](#)), driven by the DUT.

< Pulsed high by the driver while a transaction is valid.

Modport used by the UVM driver: drives the stimulus signals.

Modport used by the UVM monitor: observes every signal, drives none.

Modport used by the [extend\\_unit\\_wrapper](#) DUT: consumes the stimulus, drives the result.

## 6.5 UVM verification components

UVM classes and utilities used to verify the extend unit.

### Files

- file [reference\\_model.c](#)  
*Golden reference model for the extend unit, called from the UVM scoreboard via DPI-C.*
- file [agent.sv](#)  
*UVM agent that bundles the sequencer, driver, and monitor.*
- file [coverage.sv](#)  
*Coverage collector for extend-unit transactions.*
- file [driver.sv](#)  
*UVM driver that translates extend-unit transactions into interface signal activity.*
- file [env.sv](#)  
*UVM environment that instantiates the agent, scoreboard, and coverage collector.*
- file [extend\\_unit\\_sequence.sv](#)  
*UVM sequence that drives extend-unit transactions until coverage reaches the target.*
- file [monitor.sv](#)  
*UVM monitor that samples extend-unit transactions and publishes them to subscribers.*
- file [pkt.sv](#)  
*Transaction item used to model one extend-unit operation in the UVM environment.*
- file [scoreboard.sv](#)  
*Scoreboard that checks extend-unit transactions against the golden reference model.*
- file [sequencer.sv](#)  
*UVM sequencer that provides transactions to the driver.*
- file [test.sv](#)  
*Top-level UVM test that builds the environment and starts the main sequence.*

### 6.5.1 Detailed Description

UVM classes and utilities used to verify the extend unit.



## Chapter 7

# Directory Documentation

### 7.1 filter Directory Reference

#### 7.1.1 Detailed Description

#### 7.1.2 SystemVerilog Doxygen filter

`idv_doxyfilter_sv.pl` converts SystemVerilog into C++-like code so Doxygen can parse it.

- **Author:** Sean O'Boyle, Intelligent Design Verification
- **Source:** <https://github.com/SeanOBoyle/DoxygenFilterSystemVerilog>
- **License:** GNU GPL v3 or later (see the header of the script). The file is included here unmodified.

It is wired up through `FILTER_PATTERNS` in `../Doxyfile`; run `doxygen` from the parent directory.

### 7.2 reference\_model Directory Reference

#### Files

- file [reference\\_model.c](#)  
*Golden reference model for the extend unit, called from the UVM scoreboard via DPI-C.*

### 7.3 rtl Directory Reference

#### Files

- file [extend\\_unit.v](#)
- file [extend\\_unit\\_wrapper.sv](#)  
*Thin combinational wrapper around [extend\\_unit](#), used as the DUT.*

## 7.4 tb Directory Reference

### Files

- file [s\\_interface.sv](#)  
*Extend-unit stimulus/response interface shared by the DUT and the UVM environment.*
- file [tb\\_top.sv](#)  
*Top-level testbench that instantiates the DUT, interface, and UVM test.*

## 7.5 uvm Directory Reference

### Files

- file [agent.sv](#)  
*UVM agent that bundles the sequencer, driver, and monitor.*
- file [coverage.sv](#)  
*Coverage collector for extend-unit transactions.*
- file [driver.sv](#)  
*UVM driver that translates extend-unit transactions into interface signal activity.*
- file [env.sv](#)  
*UVM environment that instantiates the agent, scoreboard, and coverage collector.*
- file [extend\\_unit\\_pkg.sv](#)  
*Package that includes the UVM classes used by the extend-unit verification environment.*
- file [extend\\_unit\\_sequence.sv](#)  
*UVM sequence that drives extend-unit transactions until coverage reaches the target.*
- file [monitor.sv](#)  
*UVM monitor that samples extend-unit transactions and publishes them to subscribers.*
- file [pkt.sv](#)  
*Transaction item used to model one extend-unit operation in the UVM environment.*
- file [scoreboard.sv](#)  
*Scoreboard that checks extend-unit transactions against the golden reference model.*
- file [sequencer.sv](#)  
*UVM sequencer that provides transactions to the driver.*
- file [test.sv](#)  
*Top-level UVM test that builds the environment and starts the main sequence.*

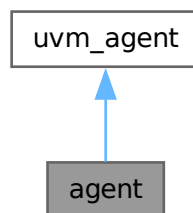
## Chapter 8

# Class Documentation

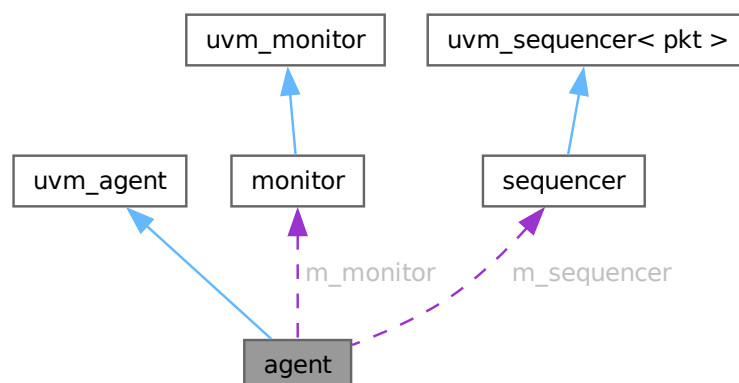
### 8.1 agent Class Reference

UVM agent that connects the sequencer, driver, and monitor.

Inheritance diagram for agent:



Collaboration diagram for agent:



## Public Member Functions

- [uvm\\_component\\_utils](#) ([agent](#)) [driver](#) m\_driver  
*Drives sequence items onto the DUT interface (null when passive).*
- [new](#) (string name="agent", uvm\_component parent=null)
- virtual void [build\\_phase](#) (uvm\_phase phase)  
*Builds the sequencer/driver (active agent only) and the monitor.*
- virtual void [connect\\_phase](#) (uvm\_phase phase)  
*Connects the driver's seq\_item port to the sequencer (active agent only).*

## Public Attributes

- [monitor](#) m\_monitor  
*Samples DUT activity and publishes it (always created).*
- [sequencer](#) m\_sequencer  
*Arbitrates sequence items for the driver (null when passive).*

### 8.1.1 Detailed Description

UVM agent that connects the sequencer, driver, and monitor.

Coordinates the active and passive verification components for the extend-unit testbench and exposes the common sequence and analysis interfaces. The sequencer and driver are only created when the agent is active; the monitor is always created.

### 8.1.2 Member Function Documentation

#### 8.1.2.1 build\_phase()

```
virtual void agent::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Builds the sequencer/driver (active agent only) and the monitor.

#### 8.1.2.2 connect\_phase()

```
virtual void agent::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the driver's seq\_item port to the sequencer (active agent only).

#### 8.1.2.3 new()

```
agent::new (  
    string name = "agent",  
    uvm_component parent = null) [inline]
```

#### 8.1.2.4 uvm\_component\_utils()

```
agent::uvm_component_utils (
    agent )
```

Drives sequence items onto the DUT interface (null when passive).

### 8.1.3 Member Data Documentation

#### 8.1.3.1 m\_monitor

```
monitor agent::m_monitor
```

Samples DUT activity and publishes it (always created).

#### 8.1.3.2 m\_sequencer

```
sequencer agent::m_sequencer
```

Arbitrates sequence items for the driver (null when passive).

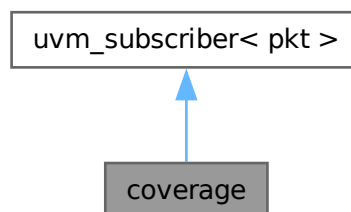
The documentation for this class was generated from the following file:

- [uvm/agent.sv](#)

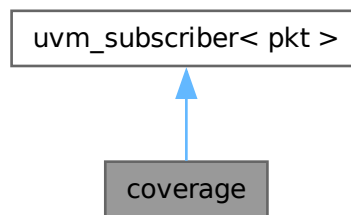
## 8.2 coverage Class Reference

UVM coverage subscriber for extend-unit transaction sampling.

Inheritance diagram for coverage:



Collaboration diagram for coverage:



## Public Member Functions

- `uvm_component_utils (coverage) pkt tr`  
*Transaction currently being sampled by `cg_extend_unit`.*
- `covergroup cg_extend_unit ()`  
*imm\_src x imm\_id-quadrant cross coverage model.*
- `new` (string name="coverage", uvm\_component parent)
- `virtual void write (pkt t)`  
*Samples the covergroup for one transaction and republishes the running coverage percentage.*

## 8.2.1 Detailed Description

UVM coverage subscriber for extend-unit transaction sampling.

Tracks `imm_src` x `imm_id`-quadrant cross coverage and publishes the current percentage to the sequence layer through the configuration database under the key "`cov_status`".

## 8.2.2 Member Function Documentation

### 8.2.2.1 cg\_extend\_unit()

```
covergroup coverage::cg_extend_unit () [inline]
```

`imm_src` x `imm_id`-quadrant cross coverage model.

One bin per `imm_src` value: 0..4 are I/S/B/J/U, 5..7 exercise the RTL's default branch.

Splits the full 25-bit `imm_id` range into 4 equal quadrants, so random stimulus hits every bin (a narrower range would make some bins statistically unreachable).

Full cross of `imm_src` with the `imm_id` quadrants.

### 8.2.2.2 new()

```
coverage::new (
    string name = "coverage",
    uvm_component parent) [inline]
```

### 8.2.2.3 uvm\_component\_utils()

```
coverage::uvm_component_utils (
    coverage )
```

Transaction currently being sampled by [cg\\_extend\\_unit](#).

### 8.2.2.4 write()

```
virtual void coverage::write (
    pkt t) [inline], [virtual]
```

Samples the covergroup for one transaction and republishes the running coverage percentage.

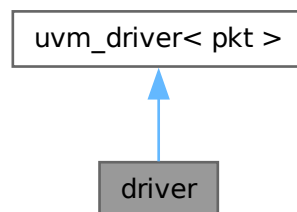
The documentation for this class was generated from the following file:

- [uvm/coverage.sv](#)

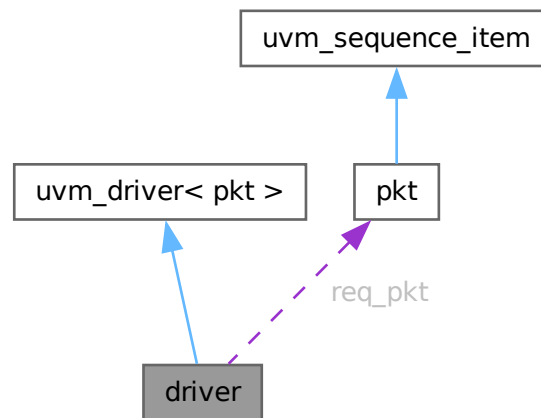
## 8.3 driver Class Reference

UVM driver for extend-unit transactions.

Inheritance diagram for driver:



Collaboration diagram for driver:



## Public Member Functions

- [uvm\\_component\\_utils](#) ([driver](#)) virtual s\_interface.driver [vif](#)  
*Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").*
- [new](#) (string name="driver", uvm\_component parent)
- virtual void [build\\_phase](#) (uvm\_phase phase)  
*Fetches the virtual interface handle set by the top-level testbench.*
- virtual task [run\\_phase](#) (uvm\_phase phase)  
*Waits for reset to be released, then drives sequence items forever.*
- virtual task [drive\\_item](#) ([pkt](#) [pkt\\_item](#))  
*Drives one immediate-format/raw-immediate pair with `enable` pulsed for one clock cycle.*

## Public Attributes

- [pkt](#) [req\\_pkt](#)  
*Sequence item currently being driven.*

### 8.3.1 Detailed Description

UVM driver for extend-unit transactions.

Converts each transaction item into the corresponding stimulus pulse on [s\\_interface](#). Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

## 8.3.2 Member Function Documentation

### 8.3.2.1 build\_phase()

```
virtual void driver::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

### 8.3.2.2 drive\_item()

```
virtual task driver::drive_item (  
    pkt pkt_item) [inline], [virtual]
```

Drives one immediate-format/raw-immediate pair with `enable` pulsed for one clock cycle.

### 8.3.2.3 new()

```
driver::new (  
    string name = "driver",  
    uvm_component parent) [inline]
```

### 8.3.2.4 run\_phase()

```
virtual task driver::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Waits for reset to be released, then drives sequence items forever.

### 8.3.2.5 uvm\_component\_utils()

```
driver::uvm_component_utils (  
    driver )
```

Driver modport of the DUT interface, obtained from the config DB (key "`vif`").

## 8.3.3 Member Data Documentation

### 8.3.3.1 req\_pkt

```
pkt driver::req_pkt
```

Sequence item currently being driven.

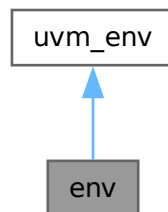
The documentation for this class was generated from the following file:

- [uvm/driver.sv](#)

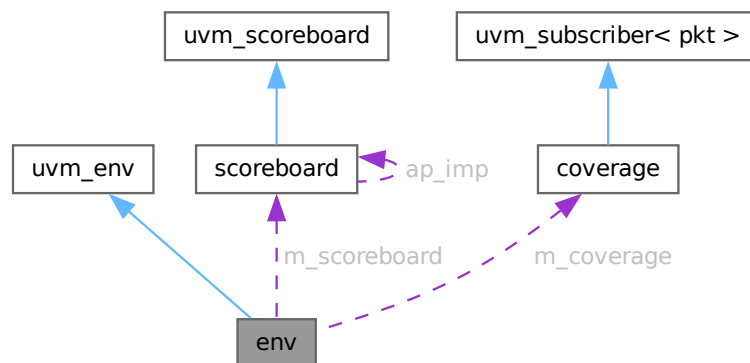
## 8.4 env Class Reference

Top-level UVM environment for the extend-unit verification bench.

Inheritance diagram for env:



Collaboration diagram for env:



### Public Member Functions

- `uvm_component_utils (env) agent m_agent`  
*Agent that drives and monitors the DUT.*
- `new (string name="env", uvm_component parent=null)`
- `virtual void build_phase (uvm_phase phase)`  
*Creates the agent, scoreboard, and coverage collector.*
- `virtual void connect_phase (uvm_phase phase)`  
*Connects the monitor's analysis port to both the scoreboard and coverage collector.*

## Public Attributes

- [scoreboard m\\_scoreboard](#)  
*Checks monitored transactions against the reference model.*
- [coverage m\\_coverage](#)  
*Collects functional coverage of monitored transactions.*

### 8.4.1 Detailed Description

Top-level UVM environment for the extend-unit verification bench.

Instantiates the agent, scoreboard, and coverage collector and connects the monitored transactions to the checkers.

### 8.4.2 Member Function Documentation

#### 8.4.2.1 build\_phase()

```
virtual void env::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the agent, scoreboard, and coverage collector.

#### 8.4.2.2 connect\_phase()

```
virtual void env::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the monitor's analysis port to both the scoreboard and coverage collector.

#### 8.4.2.3 new()

```
env::new (  
    string name = "env",  
    uvm_component parent = null) [inline]
```

#### 8.4.2.4 uvm\_component\_utils()

```
env::uvm_component_utils (  
    env )
```

Agent that drives and monitors the DUT.

### 8.4.3 Member Data Documentation

#### 8.4.3.1 m\_coverage

```
coverage env::m_coverage
```

Collects functional coverage of monitored transactions.

### 8.4.3.2 m\_scoreboard

`scoreboard env::m_scoreboard`

Checks monitored transactions against the reference model.

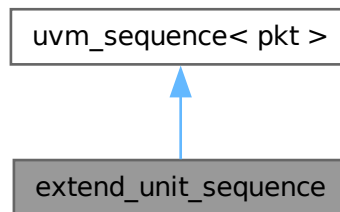
The documentation for this class was generated from the following file:

- `uvm/env.sv`

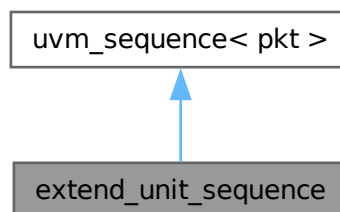
## 8.5 extend\_unit\_sequence Class Reference

UVM sequence that drives extend-unit transactions until coverage reaches the target.

Inheritance diagram for `extend_unit_sequence`:



Collaboration diagram for `extend_unit_sequence`:



### Public Member Functions

- `uvm_object_utils (extend_unit_sequence)` `real current_coverage=0`  
*Latest coverage percentage reported by the coverage collector.*
- `new` (string name="extend\_unit\_sequence")
- virtual task `body` ()  
*Sends randomized extend-unit transactions until the coverage collector reports 100%.*

## Public Attributes

- int `num_packets` = 0  
*Number of transactions sent so far in this sequence.*

### 8.5.1 Detailed Description

UVM sequence that drives extend-unit transactions until coverage reaches the target.

Repeatedly randomizes and sends a transaction, polling the coverage collector after each one, until 100% coverage is reached. Reads the coverage percentage from the configuration database key "`cov_status`".

### 8.5.2 Member Function Documentation

#### 8.5.2.1 `body()`

```
virtual task extend_unit_sequence::body () [inline], [virtual]
```

Sends randomized extend-unit transactions until the coverage collector reports 100%.

#### 8.5.2.2 `new()`

```
extend_unit_sequence::new (  
    string name = "extend_unit_sequence") [inline]
```

#### 8.5.2.3 `uvm_object_utils()`

```
extend_unit_sequence::uvm_object_utils (  
    extend_unit_sequence ) [pure virtual]
```

Latest coverage percentage reported by the coverage collector.

### 8.5.3 Member Data Documentation

#### 8.5.3.1 `num_packets`

```
int extend_unit_sequence::num_packets = 0
```

Number of transactions sent so far in this sequence.

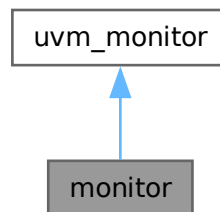
The documentation for this class was generated from the following file:

- [uvm/extend\\_unit\\_sequence.sv](#)

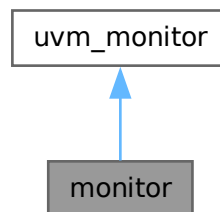
## 8.6 monitor Class Reference

UVM monitor for extend-unit transaction observation.

Inheritance diagram for monitor:



Collaboration diagram for monitor:



### Public Member Functions

- `uvm_component_utils (monitor)` virtual s\_interface.monitor `vif`  
*Monitor modport of the DUT interface, obtained from the config DB (key "`vif`").*
- `new` (string name="monitor", uvm\_component parent=null)
- virtual void `build_phase` (uvm\_phase phase)  
*Fetches the virtual interface handle set by the top-level testbench.*
- virtual task `run_phase` (uvm\_phase phase)  
*Samples one extend-unit transaction per iteration and publishes it.*

### Public Attributes

- `uvm_analysis_port< pkt > mon_analysis_port`  
*Publishes each observed transaction to the scoreboard and coverage collector.*

## 8.6.1 Detailed Description

UVM monitor for extend-unit transaction observation.

Samples the DUT-visible signals on each transfer and publishes the resulting transaction object to downstream analysis components. Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

## 8.6.2 Member Function Documentation

### 8.6.2.1 build\_phase()

```
virtual void monitor::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

### 8.6.2.2 new()

```
monitor::new (  
    string name = "monitor",  
    uvm_component parent = null) [inline]
```

### 8.6.2.3 run\_phase()

```
virtual task monitor::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Samples one extend-unit transaction per iteration and publishes it.

[extend\\_unit\\_wrapper](#) is purely combinational, but its output only settles once the DUT's [always @ \(\\*\)](#) block reacts to the new inputs, which is not guaranteed to have happened yet in the same simulation step that wakes this process on [enable](#)'s rising edge. A minimal #1 delay lets that propagation finish before the output is sampled.

### 8.6.2.4 uvm\_component\_utils()

```
monitor::uvm_component_utils (  
    monitor )
```

Monitor modport of the DUT interface, obtained from the config DB (key "[vif](#)").

## 8.6.3 Member Data Documentation

### 8.6.3.1 mon\_analysis\_port

```
uvm_analysis_port<pkt> monitor::mon_analysis_port
```

Publishes each observed transaction to the scoreboard and coverage collector.

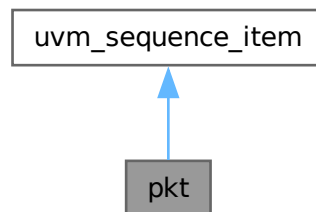
The documentation for this class was generated from the following file:

- [uvm/monitor.sv](#)

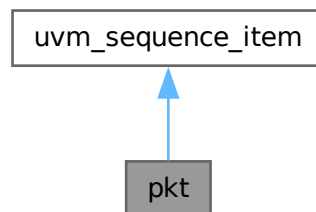
## 8.7 pkt Class Reference

Sequence item that models one extend-unit operation (stimulus + expected response fields).

Inheritance diagram for pkt:



Collaboration diagram for pkt:



### Public Member Functions

- [uvm\\_object\\_utils\\_begin](#) (pkt) [uvm\\_field\\_int](#)(i\_imm\_src\_ID
- UVM\_DEFAULT [uvm\\_field\\_int](#) (i\_imm\_ID, UVM\_DEFAULT) [uvm\\_field\\_int](#)(o\_imm\_ex\_ID
- UVM\_DEFAULT UVM\_DEFAULT [uvm\\_object\\_utils\\_end](#) [new](#) (string name="pkt")

### Public Attributes

- rand logic< 2:0 > [i\\_imm\\_src\\_ID](#)  
*Immediate format selector to drive (0..4 = I/S/B/J/U; 5..7 reserved).*
- rand logic< 24:0 > [i\\_imm\\_ID](#)  
*Raw instr[31:7] slice to drive.*
- logic< 31:0 > [o\\_imm\\_ex\\_ID](#)  
*Sign-extended immediate, sampled by the monitor.*

## 8.7.1 Detailed Description

Sequence item that models one extend-unit operation (stimulus + expected response fields).

Carries the fields randomized by the sequence/driver, and the result field filled in by the monitor for the scoreboard to check against the golden reference model.

## 8.7.2 Member Function Documentation

### 8.7.2.1 new()

```
UVM_DEFAULT UVM_DEFAULT uvm_object_utils_end pkt::new (
    string name = "pkt") [inline]
```

### 8.7.2.2 uvm\_field\_int()

```
UVM_DEFAULT pkt::uvm_field_int (
    i_imm_ID ,
    UVM_DEFAULT )
```

### 8.7.2.3 uvm\_object\_utils\_begin()

```
pkt::uvm_object_utils_begin (
    pkt )
```

## 8.7.3 Member Data Documentation

### 8.7.3.1 i\_imm\_ID

```
rand logic<24:0> pkt::i_imm_ID
```

Raw instr[31:7] slice to drive.

### 8.7.3.2 i\_imm\_src\_ID

```
rand logic<2:0> pkt::i_imm_src_ID
```

Immediate format selector to drive (0..4 = I/S/B/J/U; 5..7 reserved).

### 8.7.3.3 o\_imm\_ex\_ID

```
logic<31:0> pkt::o_imm_ex_ID
```

Sign-extended immediate, sampled by the monitor.

The documentation for this class was generated from the following file:

- [uvm/pkt.sv](#)

## 8.8 s\_interface Interface Reference

Extend-unit stimulus/response interface shared by the DUT and the UVM environment.

### 8.8.1 Detailed Description

Extend-unit stimulus/response interface shared by the DUT and the UVM environment.

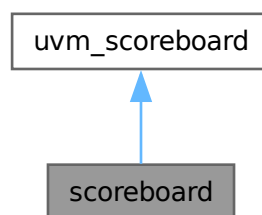
The documentation for this interface was generated from the following file:

- [tb/s\\_interface.sv](#)

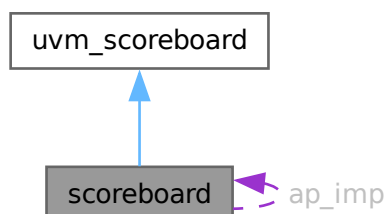
## 8.9 scoreboard Class Reference

UVM scoreboard for extend-unit functional checking.

Inheritance diagram for scoreboard:



Collaboration diagram for scoreboard:



## Public Member Functions

- `uvm_component_utils (scoreboard) uvm_analysis_imp< pkt`
- `new` (string name="scoreboard", uvm\_component parent=null)
- virtual void `build_phase` (uvm\_phase phase)  
*Creates the analysis imp that receives transactions from the monitor.*
- virtual void `write` (pkt data)  
*Analysis-port callback: checks one monitored transaction against `extend_unit_golden`.*
- virtual void `check_phase` (uvm\_phase phase)  
*Fails the test if any mismatch was recorded by `write()`.*
- virtual task `run_phase` (uvm\_phase phase)  
*Run phase; unused, checking happens in `write()` and `check_phase()`.*

## Public Attributes

- `scoreboard ap_imp`  
*Receives transactions from the monitor's analysis port.*
- int `num_errors` = 0  
*Number of mismatches found so far.*

### 8.9.1 Detailed Description

UVM scoreboard for extend-unit functional checking.

Compares monitored extend-unit inputs/outputs with the DPI-C reference model and reports mismatches as test failures.

### 8.9.2 Member Function Documentation

#### 8.9.2.1 build\_phase()

```
virtual void scoreboard::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Creates the analysis imp that receives transactions from the monitor.

#### 8.9.2.2 check\_phase()

```
virtual void scoreboard::check_phase (
    uvm_phase phase) [inline], [virtual]
```

Fails the test if any mismatch was recorded by `write()`.

#### 8.9.2.3 new()

```
scoreboard::new (
    string name = "scoreboard",
    uvm_component parent = null) [inline]
```

#### 8.9.2.4 run\_phase()

```
virtual task scoreboard::run_phase (
    uvm_phase phase) [inline], [virtual]
```

Run phase; unused, checking happens in [write\(\)](#) and [check\\_phase\(\)](#).

#### 8.9.2.5 uvm\_component\_utils()

```
scoreboard::uvm_component_utils (
    scoreboard )
```

#### 8.9.2.6 write()

```
virtual void scoreboard::write (
    pkt data) [inline], [virtual]
```

Analysis-port callback: checks one monitored transaction against [extend\\_unit\\_golden](#).

### 8.9.3 Member Data Documentation

#### 8.9.3.1 ap\_imp

```
scoreboard scoreboard::ap_imp
```

Receives transactions from the monitor's analysis port.

#### 8.9.3.2 num\_errors

```
int scoreboard::num_errors = 0
```

Number of mismatches found so far.

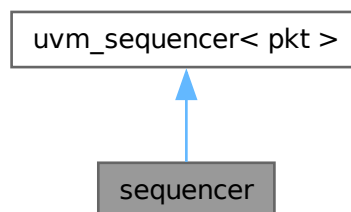
The documentation for this class was generated from the following file:

- [uvm/scoreboard.sv](#)

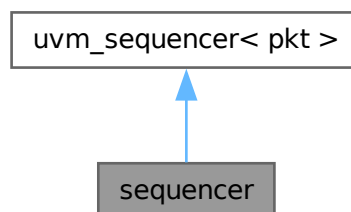
## 8.10 sequencer Class Reference

UVM sequencer for extend-unit transactions.

Inheritance diagram for sequencer:



Collaboration diagram for sequencer:



### Public Member Functions

- [uvm\\_component\\_utils](#) ([sequencer](#)) new(string name

### Public Attributes

- uvm\_component [parent](#)

### 8.10.1 Detailed Description

UVM sequencer for extend-unit transactions.

Supplies randomized sequence items to the driver during the test run.

## 8.10.2 Member Function Documentation

### 8.10.2.1 `uvm_component_utils()`

```
sequencer::uvm_component_utils (
    sequencer ) [new]
```

## 8.10.3 Member Data Documentation

### 8.10.3.1 `parent`

```
uvm_component sequencer::parent
```

#### Initial value:

```
{
    super.new (name, parent)
}
```

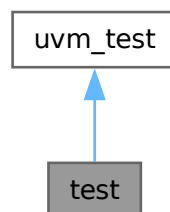
The documentation for this class was generated from the following file:

- [uvm/sequencer.sv](#)

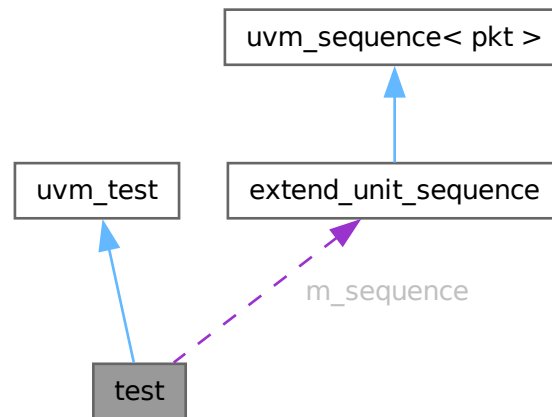
## 8.11 test Class Reference

Top-level UVM test for the extend-unit verification bench.

Inheritance diagram for test:



Collaboration diagram for test:



### Public Member Functions

- `uvm_component_utils (test) env m_env`  
*Verification environment under test.*
- `new` (string name="test", uvm\_component parent=null)
- virtual void `build_phase` (uvm\_phase phase)  
*Builds the verification environment.*
- virtual void `end_of_elaboration_phase` (uvm\_phase phase)  
*Prints the component topology once elaboration is complete.*
- virtual task `run_phase` (uvm\_phase phase)  
*Starts the coverage-driven sequence and raises/drops the run-phase objection around it.*

### Public Attributes

- `extend_unit_sequence m_sequence`  
*Main stimulus sequence, started on the agent's sequencer.*

## 8.11.1 Detailed Description

Top-level UVM test for the extend-unit verification bench.

Builds the environment, starts the main sequence, and drives the full verification flow from the test root.

## 8.11.2 Member Function Documentation

### 8.11.2.1 build\_phase()

```
virtual void test::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Builds the verification environment.

### 8.11.2.2 end\_of\_elaboration\_phase()

```
virtual void test::end_of_elaboration_phase (
    uvm_phase phase) [inline], [virtual]
```

Prints the component topology once elaboration is complete.

### 8.11.2.3 new()

```
test::new (
    string name = "test",
    uvm_component parent = null) [inline]
```

### 8.11.2.4 run\_phase()

```
virtual task test::run_phase (
    uvm_phase phase) [inline], [virtual]
```

Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

### 8.11.2.5 uvm\_component\_utils()

```
test::uvm_component_utils (
    test )
```

Verification environment under test.

## 8.11.3 Member Data Documentation

### 8.11.3.1 m\_sequence

```
extend\_unit\_sequence test::m_sequence
```

Main stimulus sequence, started on the agent's sequencer.

The documentation for this class was generated from the following file:

- [uvm/test.sv](#)

## Chapter 9

# File Documentation

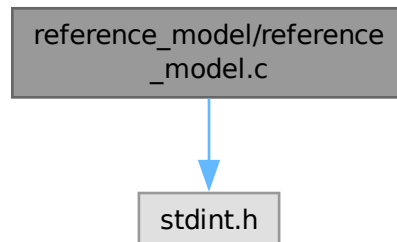
### 9.1 filter/README.md File Reference

### 9.2 reference\_model/reference\_model.c File Reference

Golden reference model for the extend unit, called from the UVM scoreboard via DPI-C.

```
#include <stdint.h>
```

Include dependency graph for reference\_model.c:



#### Enumerations

- enum `imm_src_e` {  
    `IMM_SRC_I` = 0 , `IMM_SRC_S` = 1 , `IMM_SRC_B` = 2 , `IMM_SRC_J` = 3 ,  
    `IMM_SRC_U` = 4 }

#### Functions

- void `extend_unit_golden` (uint32\_t imm\_src, uint32\_t imm\_id, uint32\_t \*exp\_imm\_ex)  
    *Computes the expected sign-extended immediate for one `imm_src` format.*

## 9.2.1 Detailed Description

Golden reference model for the extend unit, called from the UVM scoreboard via DPI-C.

Reconstructs the sign-extended RISC-V immediate for every `imm_src` format (I/S/B/J/U), operating on the same `instr[31:7]` slice (`i_imm_ID`, 25 bits) received by `extend_unit.v` after the opcode field has been stripped off by the decode stage.

## 9.2.2 Enumeration Type Documentation

### 9.2.2.1 `imm_src_e`

```
enum imm_src_e
```

#### Enumerator

|           |  |
|-----------|--|
| IMM_SRC_I |  |
| IMM_SRC_S |  |
| IMM_SRC_B |  |
| IMM_SRC_J |  |
| IMM_SRC_U |  |

## 9.2.3 Function Documentation

### 9.2.3.1 `extend_unit_golden()`

```
void extend_unit_golden (
    uint32_t imm_src,
    uint32_t imm_id,
    uint32_t * exp_imm_ex)
```

Computes the expected sign-extended immediate for one `imm_src` format.

#### Parameters

|                   |                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| <i>imm_src</i>    | Immediate format selector (see <code>imm_src_e</code> ; any other value is reserved and expected to produce 0). |
| <i>imm_id</i>     | <code>instr[31:7]</code> , right-aligned into the low 25 bits.                                                  |
| <i>exp_imm_ex</i> | Output: expected <code>o_imm_ex_ID</code> .                                                                     |

## 9.3 rtl/extend\_unit.v File Reference

### Functions

- `always (i_imm_src_ID, i_imm_ID) begin case(i_imm_src_ID) I_type`

### Variables

- input logic[2:0] `i_imm_src_ID`
- input logic[WIDTH-8:0] `i_imm_ID`
- output logic[WIDTH-1:0] `o_imm_ex_ID`
- localparam logic[2:0] `I_type` = 3'b000
- localparam logic[2:0] `S_type` = 3'b001
- localparam logic[2:0] `B_type` = 3'b010
- localparam logic[2:0] `J_type` = 3'b011
- localparam logic[2:0] `U_type` = 3'b100
- end `b0`
- end end `default {12{1'b0}}`

### 9.3.1 Function Documentation

#### 9.3.1.1 always()

```
always (
    i_imm_src_ID ,
    i_imm_ID )
```

### 9.3.2 Variable Documentation

#### 9.3.2.1 b0

```
end b0
```

#### 9.3.2.2 B\_type

```
end B_type = 3'b010
```

#### 9.3.2.3 default

```
end end default {12{1'b0}}
```

#### 9.3.2.4 i\_imm\_ID

```
end i_imm_ID
```

#### 9.3.2.5 i\_imm\_src\_ID

```
input logic [2:0] i_imm_src_ID
```

#### 9.3.2.6 I\_type

```
localparam logic [2:0] I_type = 3'b000
```

#### 9.3.2.7 J\_type

```
end J_type = 3'b011
```

#### 9.3.2.8 o\_imm\_ex\_ID

```
output logic [WIDTH-1:0] o_imm_ex_ID
```

#### 9.3.2.9 S\_type

```
end S_type = 3'b001
```

#### 9.3.2.10 U\_type

```
end U_type = 3'b100
```

## 9.4 rtl/extend\_unit\_wrapper.sv File Reference

Thin combinational wrapper around [extend\\_unit](#), used as the DUT.

### 9.4.1 Detailed Description

Thin combinational wrapper around [extend\\_unit](#), used as the DUT.

[extend\\_unit](#) has no internal state, so the wrapper simply wires the `s_interface.dut` modport straight into it, with no clock/reset logic.

## 9.5 tb/s\_interface.sv File Reference

Extend-unit stimulus/response interface shared by the DUT and the UVM environment.

## Functions

- interface [s\\_interface](#) (input logic clk, input logic reset)

### 9.5.1 Detailed Description

Extend-unit stimulus/response interface shared by the DUT and the UVM environment.

Declares the immediate-format selector/raw-immediate/extended-immediate signals and exposes the [driver](#), [monitor](#), and `dut` modports used by the UVM driver, the UVM monitor, and the [extend\\_unit\\_wrapper](#) DUT respectively.

## 9.6 tb/tb\_top.sv File Reference

Top-level testbench that instantiates the DUT, interface, and UVM test.

## Functions

- [s\\_interface vif](#) (.clk(clk),.reset(rst))
- [extend\\_unit\\_wrapper U\\_EXTEND\\_UNIT\\_WRAPPER](#) (.vif(vif))
- [run\\_test](#) ("test")

## Variables

- bit [rst](#)
- [forever](#)

### 9.6.1 Detailed Description

Top-level testbench that instantiates the DUT, interface, and UVM test.

Connects [s\\_interface](#), the [extend\\_unit\\_wrapper](#) DUT, and the UVM test entry point in a single simulation environment.

### 9.6.2 Function Documentation

#### 9.6.2.1 run\_test()

```
run_test (
    "test" )
```

#### 9.6.2.2 U\_EXTEND\_UNIT\_WRAPPER()

```
extend_unit_wrapper U_EXTEND_UNIT_WRAPPER (
    . vifvif)
```

### 9.6.2.3 vif()

```
s_interface vif (  
    . clkclk,  
    . resetrst)
```

## 9.6.3 Variable Documentation

### 9.6.3.1 forever

```
forever
```

**Initial value:**

```
{  
    rst = 0
```

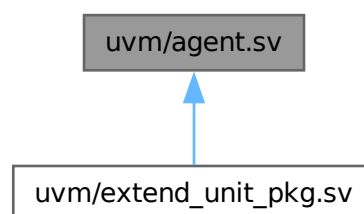
### 9.6.3.2 rst

```
bit rst
```

## 9.7 uvm/agent.sv File Reference

UVM agent that bundles the sequencer, driver, and monitor.

This graph shows which files directly or indirectly include this file:



### Classes

- class [agent](#)

*UVM agent that connects the sequencer, driver, and monitor.*

### 9.7.1 Detailed Description

UVM agent that bundles the sequencer, driver, and monitor.

Implements the standard UVM agent structure for the extend-unit verification environment, creating active and passive components as needed.

## 9.8 agent.sv

[Go to the documentation of this file.](#)

```

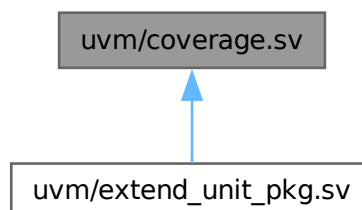
00001
00018 class agent extends uvm_agent;
00019     `uvm_component_utils (agent)
00020
00021     driver m_driver;
00022
00023     monitor m_monitor;
00024
00025     sequencer m_sequencer;
00026
00027     function new (string name = "agent", uvm_component parent = null);
00028         super.new (name, parent);
00029     endfunction
00030
00032     virtual function void build_phase (uvm_phase phase);
00033         super.build_phase (phase);
00034
00035         if (get_is_active()) begin
00036             m_sequencer = sequencer::type_id::create ("sequencer", this);
00037             m_driver = driver::type_id::create ("driver", this);
00038         end
00039
00040         m_monitor = monitor::type_id::create ("monitor", this);
00041
00042     endfunction
00043
00045     virtual function void connect_phase (uvm_phase phase);
00046         super.connect_phase (phase);
00047         if (get_is_active()) begin
00048             m_driver.seq_item_port.connect (m_sequencer.seq_item_export);
00049         end
00050     endfunction
00051
00052 endclass

```

## 9.9 uvm/coverage.sv File Reference

Coverage collector for extend-unit transactions.

This graph shows which files directly or indirectly include this file:



## Classes

- class [coverage](#)

*UVM coverage subscriber for extend-unit transaction sampling.*

### 9.9.1 Detailed Description

Coverage collector for extend-unit transactions.

Samples observed extend-unit transactions and updates the coverage model used to determine whether the verification goal has been reached.

### 9.10 coverage.sv

[Go to the documentation of this file.](#)

```

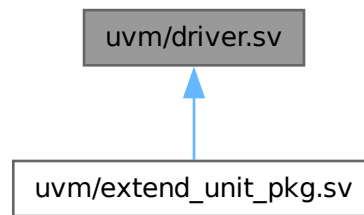
00001
00017 class coverage extends uvm_subscriber #(pkt);
00018     `uvm_component_utils(coverage)
00019
00020     pkt tr;
00021
00023     covergroup cg_extend_unit;
00024         option.per_instance = 1;
00025
00027         cp_imm_src: coverpoint tr.i_imm_src_ID {
00028             bins formats[8] = {[0:7]};
00029         }
00030
00033         cp_imm_id: coverpoint tr.i_imm_ID {
00034             bins low      = {[25'h000_0000:25'h07F_FFFF]};
00035             bins low_med  = {[25'h080_0000:25'h0FF_FFFF]};
00036             bins high_med = {[25'h100_0000:25'h17F_FFFF]};
00037             bins high     = {[25'h180_0000:25'h1FF_FFFF]};
00038         }
00039
00041         cp_cross: cross cp_imm_src, cp_imm_id;
00042
00043     endgroup
00044
00045     function new(string name = "coverage", uvm_component parent);
00046         super.new(name, parent);
00047         cg_extend_unit = new();
00048         cg_extend_unit.set_inst_name("extend_unit_cov");
00049     endfunction
00050
00052     virtual function void write(pkt t);
00053         this.tr = t;
00054         cg_extend_unit.sample();
00055         uvm_config_db#(real)::set(null, "*", "cov_status", cg_extend_unit.get_inst_coverage());
00056     endfunction
00057
00058 endclass

```

### 9.11 uvm/driver.sv File Reference

UVM driver that translates extend-unit transactions into interface signal activity.

This graph shows which files directly or indirectly include this file:



## Classes

- class `driver`

*UVM driver for extend-unit transactions.*

### 9.11.1 Detailed Description

UVM driver that translates extend-unit transactions into interface signal activity.

Receives sequence items from the sequencer and drives the immediate format/raw immediate onto `s_interface` for one clock cycle each.

## 9.12 driver.sv

[Go to the documentation of this file.](#)

```

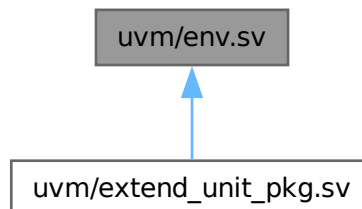
00001
00017 class driver extends uvm_driver #(pkt);
00018     `uvm_component_utils (driver)
00019
00020     virtual s_interface.driver vif;
00021
00022     pkt req_pkt;
00023
00024     function new (string name = "driver", uvm_component parent);
00025         super.new(name, parent);
00026     endfunction
00027
00029     virtual function void build_phase (uvm_phase phase);
00030         super.build_phase(phase);
00031         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00032             `uvm_fatal (get_type_name (), "Didn't get handle to virtual interface if_name")
00033         end
00034     endfunction
00035
00037     virtual task run_phase(uvm_phase phase);
00038         super.run_phase(phase);
00039
00040         wait (vif.reset == 1'b1);
00041
00042         forever begin
00043             seq_item_port.get_next_item(req_pkt);
00044             drive_item(req_pkt);
00045             seq_item_port.item_done();
00046         end
00047     endtask
  
```

```
00048
00050     virtual task drive_item (pkt pkt_item);
00051         @(posedge vif.clk);
00052         vif.imm_src <= pkt_item.i_imm_src_ID;
00053         vif.imm_id  <= pkt_item.i_imm_ID;
00054         vif.enable  <= '1;
00055
00056         @(posedge vif.clk);
00057         vif.enable  <= '0;
00058     endtask
00059
00060 endclass
```

## 9.13 uvm/env.sv File Reference

UVM environment that instantiates the agent, scoreboard, and coverage collector.

This graph shows which files directly or indirectly include this file:



### Classes

- class [env](#)  
*Top-level UVM environment for the extend-unit verification bench.*

### 9.13.1 Detailed Description

UVM environment that instantiates the agent, scoreboard, and coverage collector.

Connects the main verification components so that transactions generated by the agent can be checked and tracked by the scoreboard and coverage.

## 9.14 env.sv

[Go to the documentation of this file.](#)

```

00001
00016 class env extends uvm_env;
00017     `uvm_component_utils (env)
00018
00019     agent m_agent;
00020
00021     scoreboard m_scoreboard;
00022
00023     coverage m_coverage;
00024
00025     function new (string name = "env", uvm_component parent = null);
00026         super.new (name, parent);
00027     endfunction
00028
00030     virtual function void build_phase (uvm_phase phase);
00031         super.build_phase (phase);
00032         m_agent = agent::type_id::create ("agent", this);
00033         m_scoreboard = scoreboard::type_id::create ("scoreboard", this);
00034         m_coverage = coverage::type_id::create ("coverage", this);
00035     endfunction
00036
00038     virtual function void connect_phase (uvm_phase phase);
00039         super.connect_phase (phase);
00040         m_agent.m_monitor.mon_analysis_port.connect (m_scoreboard.ap_imp);
00041         m_agent.m_monitor.mon_analysis_port.connect (m_coverage.analysis_export);
00042     endfunction
00043
00044 endclass

```

## 9.15 uvm/extend\_unit\_pkg.sv File Reference

Package that includes the UVM classes used by the extend-unit verification environment.

```

#include "uvm_macros.svh"
#include "pkt.sv"
#include "sequencer.sv"
#include "extend_unit_sequence.sv"
#include "driver.sv"
#include "monitor.sv"
#include "agent.sv"
#include "coverage.sv"
#include "scoreboard.sv"
#include "env.sv"
#include "test.sv"

```

Include dependency graph for extend\_unit\_pkg.sv:



### 9.15.1 Detailed Description

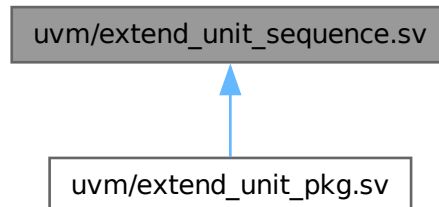
Package that includes the UVM classes used by the extend-unit verification environment.

Centralizes the inclusion of all verification components required to build the extend-unit UVM testbench.

## 9.16 uvm/extend\_unit\_sequence.sv File Reference

UVM sequence that drives extend-unit transactions until coverage reaches the target.

This graph shows which files directly or indirectly include this file:



### Classes

- class [extend\\_unit\\_sequence](#)

*UVM sequence that drives extend-unit transactions until coverage reaches the target.*

### 9.16.1 Detailed Description

UVM sequence that drives extend-unit transactions until coverage reaches the target.

Repeatedly sends randomized transactions while waiting for the coverage collector to report completion of the sampling process.

## 9.17 extend\_unit\_sequence.sv

[Go to the documentation of this file.](#)

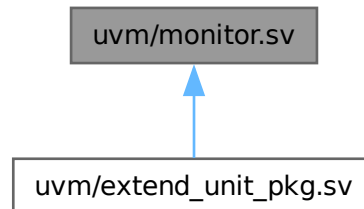
```

00001
00017 class extend_unit_sequence extends uvm_sequence #(pkt);
00018     `uvm_object_utils(extend_unit_sequence)
00019
00020     real current_coverage = 0;
00021
00022     int num_packets = 0;
00023
00024     function new (string name = "extend_unit_sequence");
00025         super.new(name);
00026     endfunction
00027
00029     virtual task body();
00030         pkt packet;
00031         `uvm_create(packet)
00032         while (current_coverage < 100.0) begin
00033             `uvm_rand_send(packet)
00034             num_packets++;
00035             void' (uvm_config_db#(real)::get(null, "*", "cov_status", current_coverage));
00036             `uvm_info("SEQ", $sformatf("Status: %0.2f%%", current_coverage), UVM_LOW)
00037         end
00038         `uvm_info("SEQ", $sformatf("Total packets sent: %0d", num_packets), UVM_LOW)
00039     endtask
00040
00041 endclass
  
```

## 9.18 uvm/monitor.sv File Reference

UVM monitor that samples extend-unit transactions and publishes them to subscribers.

This graph shows which files directly or indirectly include this file:



### Classes

- class [monitor](#)

*UVM monitor for extend-unit transaction observation.*

### 9.18.1 Detailed Description

UVM monitor that samples extend-unit transactions and publishes them to subscribers.

Observes [s\\_interface](#) for each transaction and forwards the completed imm\_src/imm\_id/imm\_ex fields to the scoreboard and coverage collector.

## 9.19 monitor.sv

[Go to the documentation of this file.](#)

```

00001
00018 class monitor extends uvm_monitor;
00019     `uvm_component_utils(monitor)
00020
00021     virtual s_interface.monitor vif;
00022
00023     uvm_analysis_port #(pkt) mon_analysis_port;
00024
00025     function new(string name = "monitor", uvm_component parent = null);
00026         super.new(name, parent);
00027     endfunction
00028
00030     virtual function void build_phase(uvm_phase phase);
00031         super.build_phase(phase);
00032         mon_analysis_port = new("mon_analysis_port", this);
00033
00034         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00035             `uvm_error(get_type_name(), "DUT interface not found")
00036         end
00037     endfunction
00038
00048     virtual task run_phase(uvm_phase phase);
00049         super.run_phase(phase);
  
```

```

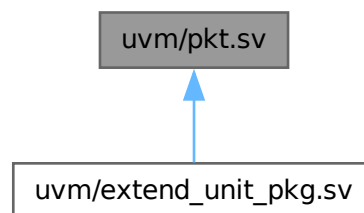
00050
00051     forever begin
00052         pkt mon_pkt;
00053         mon_pkt = pkt::type_id::create("mon_pkt", this);
00054
00055         @(posedge vif.enable);
00056         #1;
00057
00058         mon_pkt.i_imm_src_ID = vif.imm_src;
00059         mon_pkt.i_imm_ID    = vif.imm_id;
00060         mon_pkt.o_imm_ex_ID = vif.imm_ex;
00061
00062         `uvm_info(get_type_name(),
00063             $sformatf("Monitored IMM_SRC=%0d, IMM_ID=%0h, IMM_EX=%0h",
00064                 mon_pkt.i_imm_src_ID, mon_pkt.i_imm_ID, mon_pkt.o_imm_ex_ID),
00065                 UVM_LOW)
00066
00067         mon_analysis_port.write(mon_pkt);
00068     end
00069 endtask
00070
00071 endclass

```

## 9.20 uvm/pkt.sv File Reference

Transaction item used to model one extend-unit operation in the UVM environment.

This graph shows which files directly or indirectly include this file:



### Classes

- class `pkt`

*Sequence item that models one extend-unit operation (stimulus + expected response fields).*

### 9.20.1 Detailed Description

Transaction item used to model one extend-unit operation in the UVM environment.

Encapsulates the immediate-format selector and raw immediate bits driven into the extend unit, plus the extended immediate sampled back from it.

## 9.21 pkt.sv

[Go to the documentation of this file.](#)

```

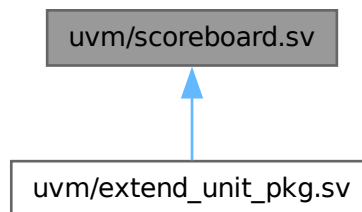
00001
00017 class pkt extends uvm_sequence_item;
00018
00019     rand logic [2:0] i_imm_src_ID;
00020     rand logic [24:0] i_imm_ID;
00021     logic [31:0] o_imm_ex_ID;
00022
00023     `uvm_object_utils_begin(pkt)
00024         `uvm_field_int (i_imm_src_ID, UVM_DEFAULT)
00025         `uvm_field_int (i_imm_ID, UVM_DEFAULT)
00026         `uvm_field_int (o_imm_ex_ID, UVM_DEFAULT)
00027     `uvm_object_utils_end
00028
00029     function new(string name = "pkt");
00030         super.new(name);
00031     endfunction
00032
00033 endclass

```

## 9.22 uvm/scoreboard.sv File Reference

Scoreboard that checks extend-unit transactions against the golden reference model.

This graph shows which files directly or indirectly include this file:



### Classes

- class [scoreboard](#)  
*UVM scoreboard for extend-unit functional checking.*

### Functions

- void [extend\\_unit\\_golden](#) (input int unsigned imm\_src, input int unsigned imm\_id, output int unsigned exp\_↔ imm\_ex)  
*DPI-C import of the golden reference model (see [reference\\_model.c](#)).*

### 9.22.1 Detailed Description

Scoreboard that checks extend-unit transactions against the golden reference model.

Receives monitored transactions and validates them against the DPI-C reference model implemented in the C source file.

### 9.22.2 Function Documentation

#### 9.22.2.1 `extend_unit_golden()`

```
void extend_unit_golden (
    input int unsigned imm_src,
    input int unsigned imm_id,
    output int unsigned exp_imm_ex)
```

DPI-C import of the golden reference model (see [reference\\_model.c](#)).

#### Parameters

|                   |                                        |
|-------------------|----------------------------------------|
| <i>imm_src</i>    | Immediate format selector.             |
| <i>imm_id</i>     | Raw instr[31:7] slice.                 |
| <i>exp_imm_ex</i> | Expected <a href="#">o_imm_ex_ID</a> . |

## 9.23 scoreboard.sv

[Go to the documentation of this file.](#)

```
00001
00016 import "DPI-C" function void extend_unit_golden(
00017     input  int unsigned imm_src,
00018     input  int unsigned imm_id,
00019     output int unsigned exp_imm_ex
00020 );
00021
00029 class scoreboard extends uvm_scoreboard;
00030     `uvm_component_utils(scoreboard)
00031
00032     uvm_analysis_imp #(pkt, scoreboard) ap_imp;
00033
00034     int num_errors = 0;
00035
00036     function new(string name = "scoreboard", uvm_component parent = null);
00037         super.new(name, parent);
00038     endfunction
00039
00041     virtual function void build_phase(uvm_phase phase);
00042         super.build_phase(phase);
00043         ap_imp = new("ap_imp", this);
00044     endfunction
00045
00047     virtual function void write(pkt data);
00048         logic [31:0] exp_imm_ex;
00049
00050         extend_unit_golden(data.i_imm_src_ID, data.i_imm_ID, exp_imm_ex);
00051
00052         if (data.o_imm_ex_ID == exp_imm_ex) begin
00053             `uvm_info("SCOREBOARD",
00054                 $sformatf("PASS: imm_src=%0d imm_id=%0h imm_ex=%0h",
00055                     data.i_imm_src_ID, data.i_imm_ID, data.o_imm_ex_ID),
00056                     UVM_LOW)
00057         end
00058         else begin
00059             `uvm_error("SCOREBOARD",
```

```

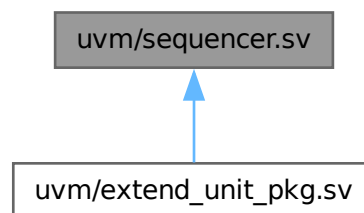
00060         $sformatf("FAIL: imm_src=%0d imm_id=%0h EXPECTED imm_ex=%0h GOT imm_ex=%0h",
00061             data.i_imm_src_ID, data.i_imm_ID, exp_imm_ex, data.o_imm_ex_ID))
00062         this.num_errors++;
00063     end
00064 endfunction
00065
00067 virtual function void check_phase(uvm_phase phase);
00068     super.check_phase(phase);
00069     if (this.num_errors > 0) begin
00070         `uvm_fatal("FINAL_RESULT",
00071             $sformatf("TEST FAILED: Scoreboard found %0d errors.", num_errors))
00072     end
00073     else begin
00074         `uvm_info("FINAL_RESULT", "TEST PASS: All transactions were correct.", UVM_NONE)
00075     end
00076 endfunction
00077
00079 virtual task run_phase(uvm_phase phase);
00080     super.run_phase(phase);
00081 endtask
00082
00083 endclass

```

## 9.24 uvm/sequencer.sv File Reference

UVM sequencer that provides transactions to the driver.

This graph shows which files directly or indirectly include this file:



### Classes

- class `sequencer`  
*UVM sequencer for extend-unit transactions.*

### 9.24.1 Detailed Description

UVM sequencer that provides transactions to the driver.

Acts as the transaction source for the UVM driver in the extend-unit verification.

## 9.25 sequencer.sv

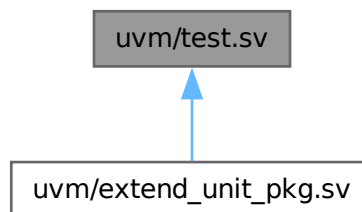
[Go to the documentation of this file.](#)

```
00001
00014 class sequencer extends uvm_sequencer #(pkt);
00015
00016     `uvm_component_utils (sequencer)
00017
00018     function new (string name="sequencer", uvm_component parent);
00019         super.new (name, parent);
00020     endfunction
00021
00022 endclass
```

## 9.26 uvm/test.sv File Reference

Top-level UVM test that builds the environment and starts the main sequence.

This graph shows which files directly or indirectly include this file:



### Classes

- class [test](#)  
*Top-level UVM test for the extend-unit verification bench.*

### 9.26.1 Detailed Description

Top-level UVM test that builds the environment and starts the main sequence.

Instantiates the verification environment and launches the extend-unit sequence that drives the DUT under test.

## 9.27 test.sv

[Go to the documentation of this file.](#)

```
00001
00016 class test extends uvm_test;
00017     `uvm_component_utils (test)
00018
00019     env m_env;
00020
00021     extend_unit_sequence m_sequence;
00022
00023     function new (string name = "test", uvm_component parent = null);
00024         super.new (name, parent);
00025     endfunction
00026
00028     virtual function void build_phase (uvm_phase phase);
00029         super.build_phase (phase);
00030         m_env = env::type_id::create ("env", this);
00031     endfunction
00032
00034     virtual function void end_of_elaboration_phase (uvm_phase phase);
00035         uvm_root::get().print_topology();
00036     endfunction
00037
00039     virtual task run_phase(uvm_phase phase);
00040         super.run_phase(phase);
00041
00042         m_sequence = extend_unit_sequence::type_id::create("sequence");
00043
00044         phase.raise_objection(this);
00045         m_sequence.start(m_env.m_agent.m_sequencer);
00046         phase.drop_objection(this);
00047     endtask
00048 endclass
```



# Index

agent, [17](#)  
    build\_phase, [18](#)  
    connect\_phase, [18](#)  
    m\_monitor, [19](#)  
    m\_sequencer, [19](#)  
    new, [18](#)  
    uvm\_component\_utils, [18](#)  
always  
    extend\_unit.v, [41](#)  
ap\_imp  
    scoreboard, [34](#)  
  
b0  
    extend\_unit.v, [41](#)  
B\_type  
    extend\_unit.v, [41](#)  
body  
    extend\_unit\_sequence, [27](#)  
build\_phase  
    agent, [18](#)  
    driver, [23](#)  
    env, [25](#)  
    monitor, [29](#)  
    scoreboard, [33](#)  
    test, [37](#)  
  
cg\_extend\_unit  
    coverage, [20](#)  
check\_phase  
    scoreboard, [33](#)  
connect\_phase  
    agent, [18](#)  
    env, [25](#)  
coverage, [19](#)  
    cg\_extend\_unit, [20](#)  
    new, [20](#)  
    uvm\_component\_utils, [21](#)  
    write, [21](#)  
  
default  
    extend\_unit.v, [41](#)  
drive\_item  
    driver, [23](#)  
driver, [21](#)  
    build\_phase, [23](#)  
    drive\_item, [23](#)  
    new, [23](#)  
    req\_pkt, [23](#)  
    run\_phase, [23](#)  
    uvm\_component\_utils, [23](#)  
  
end\_of\_elaboration\_phase  
    test, [37](#)  
env, [24](#)  
    build\_phase, [25](#)  
    connect\_phase, [25](#)  
    m\_coverage, [25](#)  
    m\_scoreboard, [25](#)  
    new, [25](#)  
    uvm\_component\_utils, [25](#)  
extend\_unit.v  
    always, [41](#)  
    b0, [41](#)  
    B\_type, [41](#)  
    default, [41](#)  
    i\_imm\_ID, [41](#)  
    i\_imm\_src\_ID, [41](#)  
    I\_type, [42](#)  
    J\_type, [42](#)  
    o\_imm\_ex\_ID, [42](#)  
    S\_type, [42](#)  
    U\_type, [42](#)  
extend\_unit\_golden  
    reference\_model.c, [40](#)  
    scoreboard.sv, [54](#)  
extend\_unit\_sequence, [26](#)  
    body, [27](#)  
    new, [27](#)  
    num\_packets, [27](#)  
    uvm\_object\_utils, [27](#)  
  
filter Directory Reference, [15](#)  
filter/README.md, [39](#)  
forever  
    tb\_top.sv, [44](#)  
  
i\_imm\_ID  
    extend\_unit.v, [41](#)  
    pkt, [31](#)  
i\_imm\_src\_ID  
    extend\_unit.v, [41](#)  
    pkt, [31](#)  
I\_type  
    extend\_unit.v, [42](#)  
IMM\_SRC\_B  
    reference\_model.c, [40](#)  
imm\_src\_e  
    reference\_model.c, [40](#)  
IMM\_SRC\_I  
    reference\_model.c, [40](#)  
IMM\_SRC\_J

- reference\_model.c, 40
- IMM\_SRC\_S
  - reference\_model.c, 40
- IMM\_SRC\_U
  - reference\_model.c, 40
- Interfaces, 12
  - s\_interface, 12
- J\_type
  - extend\_unit.v, 42
- m\_coverage
  - env, 25
- m\_monitor
  - agent, 19
- m\_scoreboard
  - env, 25
- m\_sequence
  - test, 38
- m\_sequencer
  - agent, 19
- Modules, 11
- mon\_analysis\_port
  - monitor, 29
- monitor, 28
  - build\_phase, 29
  - mon\_analysis\_port, 29
  - new, 29
  - run\_phase, 29
  - uvm\_component\_utils, 29
- new
  - agent, 18
  - coverage, 20
  - driver, 23
  - env, 25
  - extend\_unit\_sequence, 27
  - monitor, 29
  - pkt, 31
  - scoreboard, 33
  - test, 38
- num\_errors
  - scoreboard, 34
- num\_packets
  - extend\_unit\_sequence, 27
- o\_imm\_ex\_ID
  - extend\_unit.v, 42
  - pkt, 31
- parent
  - sequencer, 36
- pkt, 30
  - i\_imm\_ID, 31
  - i\_imm\_src\_ID, 31
  - new, 31
  - o\_imm\_ex\_ID, 31
  - uvm\_field\_int, 31
  - uvm\_object\_utils\_begin, 31
- reference\_model Directory Reference, 15
- reference\_model.c
  - extend\_unit\_golden, 40
  - IMM\_SRC\_B, 40
  - imm\_src\_e, 40
  - IMM\_SRC\_I, 40
  - IMM\_SRC\_J, 40
  - IMM\_SRC\_S, 40
  - IMM\_SRC\_U, 40
- reference\_model/reference\_model.c, 39
- req\_pkt
  - driver, 23
- rst
  - tb\_top.sv, 44
- RTL design, 11
- rtl Directory Reference, 15
- rtl/extend\_unit.v, 41
- rtl/extend\_unit\_wrapper.sv, 42
- run\_phase
  - driver, 23
  - monitor, 29
  - scoreboard, 33
  - test, 38
- run\_test
  - tb\_top.sv, 43
- s\_interface, 32
  - Interfaces, 12
- S\_type
  - extend\_unit.v, 42
- scoreboard, 32
  - ap\_imp, 34
  - build\_phase, 33
  - check\_phase, 33
  - new, 33
  - num\_errors, 34
  - run\_phase, 33
  - uvm\_component\_utils, 34
  - write, 34
- scoreboard.sv
  - extend\_unit\_golden, 54
- sequencer, 35
  - parent, 36
  - uvm\_component\_utils, 36
- tb Directory Reference, 16
- tb/s\_interface.sv, 42
- tb/tb\_top.sv, 43
- tb\_top.sv
  - forever, 44
  - rst, 44
  - run\_test, 43
  - U\_EXTEND\_UNIT\_WRAPPER, 43
  - vif, 43
- test, 36
  - build\_phase, 37
  - end\_of\_elaboration\_phase, 37
  - m\_sequence, 38
  - new, 38

- run\_phase, [38](#)
- uvm\_component\_utils, [38](#)
- Testbench components, [11](#)
- U\_EXTEND\_UNIT\_WRAPPER
  - tb\_top.sv, [43](#)
- U\_type
  - extend\_unit.v, [42](#)
- uvm Directory Reference, [16](#)
- UVM verification components, [12](#)
- uvm/agent.sv, [44](#), [45](#)
- uvm/coverage.sv, [45](#), [46](#)
- uvm/driver.sv, [46](#), [47](#)
- uvm/env.sv, [48](#), [49](#)
- uvm/extend\_unit\_pkg.sv, [49](#)
- uvm/extend\_unit\_sequence.sv, [50](#)
- uvm/monitor.sv, [51](#)
- uvm/pkt.sv, [52](#), [53](#)
- uvm/scoreboard.sv, [53](#), [54](#)
- uvm/sequencer.sv, [55](#), [56](#)
- uvm/test.sv, [56](#), [57](#)
- uvm\_component\_utils
  - agent, [18](#)
  - coverage, [21](#)
  - driver, [23](#)
  - env, [25](#)
  - monitor, [29](#)
  - scoreboard, [34](#)
  - sequencer, [36](#)
  - test, [38](#)
- uvm\_field\_int
  - pkt, [31](#)
- uvm\_object\_utils
  - extend\_unit\_sequence, [27](#)
- uvm\_object\_utils\_begin
  - pkt, [31](#)
- vif
  - tb\_top.sv, [43](#)
- write
  - coverage, [21](#)
  - scoreboard, [34](#)